BLOC-BOT

Wassim Chetioui Seif Ali Charad

October 25, 2026

Contents

2 Study and design **2**
2.1 Introduction . . . 2
2.2 Specification . . . 2
2.2.1 Origin of the need . . . 2
2.2.2 Target users . . . 3
2.2.3 Functional requirements . . . 3
2.2.4 Non-functional requirements and constraints . . . 3
2.3 System architecture . . . 4
2.4 The robot . . . 4
2.4.1 Microcontroller: ESP32-S3 (N16R8) . . . 4
2.4.2 Motor driver: DRV8833 . . . 6
2.4.3 Motors: TT gearmotors . . . 8
2.4.4 Wheel speed sensors: LM393 slotted encoders . . . 9
2.4.5 Ultrasonic sensor: HC-SR04 . . . 11
2.4.6 Inertial sensor: MPU6050 . . . 12
2.4.7 Power supply . . . 13
2.5 The controller . . . 14
2.5.1 Microcontroller: ESP32 DevKit . . . 14
2.5.2 Human interface: joystick, buttons and display . . . 15
2.5.3 Power supply . . . 16
2.6 Wireless link: ESP-NOW . . . 16
2.7 The modular platform . . . 18
2.8 Conclusion . . . 18

3 Realisation and tests **19**
3.1 Introduction . . . 19
3.2 Wiring . . . 19
3.3 Firmware . . . 22
3.3.1 Tools . . . 22
3.3.2 Communication protocol . . . 23
3.3.3 Robot firmware . . . 23
3.3.4 Controller firmware . . . 25
3.4 Problems met and solutions . . . 26
3.5 Tests and results . . . 26
3.6 Cost . . . 27
3.7 Conclusion . . . 27

Chapter 2

Study and design

2.1 Introduction

Plan: Half a page. What this chapter does: turn the problem from the general introduction into a specification, show the system architecture, then present every electronic component with its working principle and the reason it was chosen. The modular platform comes last.

2.2 Specification

2.2.1 Origin of the need

BLOC-BOT grew out of our own experience of student robotics competitions. Our team took part in two consecutive editions of the Algerian Robot Cup (ARC 3 and ARC 4), and in several other competitions, without success. Each ARC edition has two stages. To qualify, a team's robot must follow a line through a maze, and the fastest robots go through. The main event then calls for a second robot, built for a task that changes from one year to the next. Looking back, we traced most of our difficulties to two causes.

The first was time. For every competition we built the robot's modules from scratch: a new chassis, new sensor mounts, new wiring, and in the case of ARC, two robots instead of one. Each took long enough that we reached the competition underprepared, with too little time left for testing and tuning.

The second was the specification itself. Organisers often announce the rules late or in vague terms. Unsure of what would be asked, we tended to over-prepare and build a robot more complicated than the task required. A more complicated robot has more ways to fail, and some of ours failed during the competition itself.

Both problems point to the same answer: a single robot that is ready in advance and can be reconfigured quickly for each competition, rather than a new robot every time. Most competitions in this category rely on a small, recurring set of sensors (line sensors, distance sensors, wheel encoders). If each of these has a standard way of being mounted and wired, adapting the robot becomes a matter of choosing modules instead of designing them. This is the idea behind BLOC-BOT and its interchangeable mechanical holders, which we call *blocs*.

2.2.2 Target users

The platform is aimed first at students and teams preparing for robotics competitions, who need a working base quickly and want to spend their time on strategy and tuning rather than on rebuilding mechanics. Because the hardware and firmware are open and the platform is designed to be extended, it is also meant for students and researchers who need a small mobile robot as a test bench for their own sensors or algorithms.

2.2.3 Functional requirements

Table 2.1 lists what the system must do. The last column gives the state of each requirement at the time of writing, so that the specification can be read against the results in Chapter 3.

Table 2.1: Functional requirements of BLOC-BOT and their state at the time of writing.

ID	Requirement	State
FR1	The robot is driven by hand from a handheld controller over a wireless link, with no computer or phone in the loop.	Done
FR2	The controller shows feedback from the robot, starting with the speed of its wheels.	Done
FR3	The robot stops on its own when it no longer receives commands from the controller.	Done
FR4	Sensors and modules are fixed to the robot through a standard hole pattern and printed holders, without redesigning the chassis.	Done
FR5	The controller runs from its own rechargeable battery.	Done
FR6	The robot follows a line using a multi-channel infrared sensor.	In progress
FR7	The robot detects obstacles and parks itself using distance sensors.	Planned

2.2.4 Non-functional requirements and constraints

Besides what the system does, four conditions shaped how it was built.

- **Local availability of components.** Most of the components were chosen because they could be bought in Algeria or were already in our hands, not after a comparison of every alternative on the market. We state this plainly here, and each component section in this chapter says when availability was the deciding reason.
- **Low cost.** The platform is meant for students, so it is built from common hobbyist modules and 3D-printed parts. Its cost is detailed in Chapter 3.
- **Reproducibility.** The mechanical parts are printed on an ordinary desktop fused deposition modelling (FDM) printer, and the hardware designs and firmware are to be published as open source.
- **Size.** The size is set by the waffle plate, a 210×210 mm printed plate that carries every other part. In the CAD model of the assembled robot, the wheels stand out on each side, giving an overall width of 242 mm and a length of 214 mm. The robot is 102 mm tall up to its upper plate and 133 mm with the ultrasonic sensor mounted on top (values measured in the Fusion assembly, version 13).

2.3 System architecture

BLOC-BOT is made of two separate boards that share no wire: a handheld controller and the robot itself. Each has its own microcontroller and its own battery, and the two talk only over a wireless ESP-NOW link (Section 2.6). Figure 2.1 shows both, with the main signals between their parts.

The **controller** is the human side of the system. It reads the joystick and the four buttons, shows a menu and the robot's state on a small OLED screen, and sends drive commands to the robot every 50 ms while its Drive screen is open.

The **robot** turns each drive command into a speed for each wheel and drives the two TT gearmotors through a DRV8833 dual H-bridge. Two LM393 slotted sensors count the turns of the wheels, and every 100 ms the robot sends the measured wheel speeds back to the controller, which displays them. If drive commands stop arriving, the robot stops its motors on its own, which meets requirement FR3. The robot also carries an HC-SR04 ultrasonic sensor at its front, which measures the distance to obstacles, and an MPU6050 inertial sensor, which measures its tilt and rotation. Both readings are sent to the controller, the distance every 100 ms and the tilt every 50 ms, and shown on their own screens.

The two power chains are independent. The robot runs from a 2S lithium-ion pack that feeds the motor driver directly and the microcontroller through a 5 V step-down converter. The controller runs from a single lithium-ion cell with its own charger and a 5 V step-up converter. Both are detailed in Sections 2.4 and 2.5.

2.4 The robot

2.4.1 Microcontroller: ESP32-S3 (N16R8)

The robot is controlled by an ESP32-S3, a system on a chip (SoC) from Espressif that combines a microcontroller with a 2.4 GHz radio. Its main features are summarised in Table 2.2. The chip has a dual-core Xtensa LX7 processor running at up to 240 MHz, 512 KB of internal SRAM, and both Wi-Fi (IEEE 802.11b/g/n) and Bluetooth 5 Low Energy [1]. The radio is what makes the wireless link of Section 2.6 possible without any extra module.

The chip is not used on its own but inside a module marked ESP-32-S3-N16R8. This is a third-party module rather than an Espressif one, but it follows the pinout and the naming of Espressif's ESP32-S3-WROOM-1 [2]: N16R8 stands for 16 MB of flash memory and 8 MB of octal pseudo-static RAM (PSRAM). The module adds these memories, the crystal and a printed antenna, and exposes 36 general-purpose input/output (GPIO) pins. It sits on a development board laid out like Espressif's ESP32-S3-DevKitC-1, which brings the pins out to two headers, carries two USB-C ports for programming and power, and a regulator that turns a 5 V supply into the 3.3 V the module needs [3]. On the robot, that 5 V comes from the step-down converter described in Section 2.4.7.

Why the ESP32-S3. The robot is meant to grow: every new bloc brings a sensor, and every sensor needs pins. The ESP32-S3 was chosen for its number of usable GPIOs, which leaves room for the line sensor, the ultrasonic sensor and later additions without changing the microcontroller. Its PSRAM is not used by the current firmware.

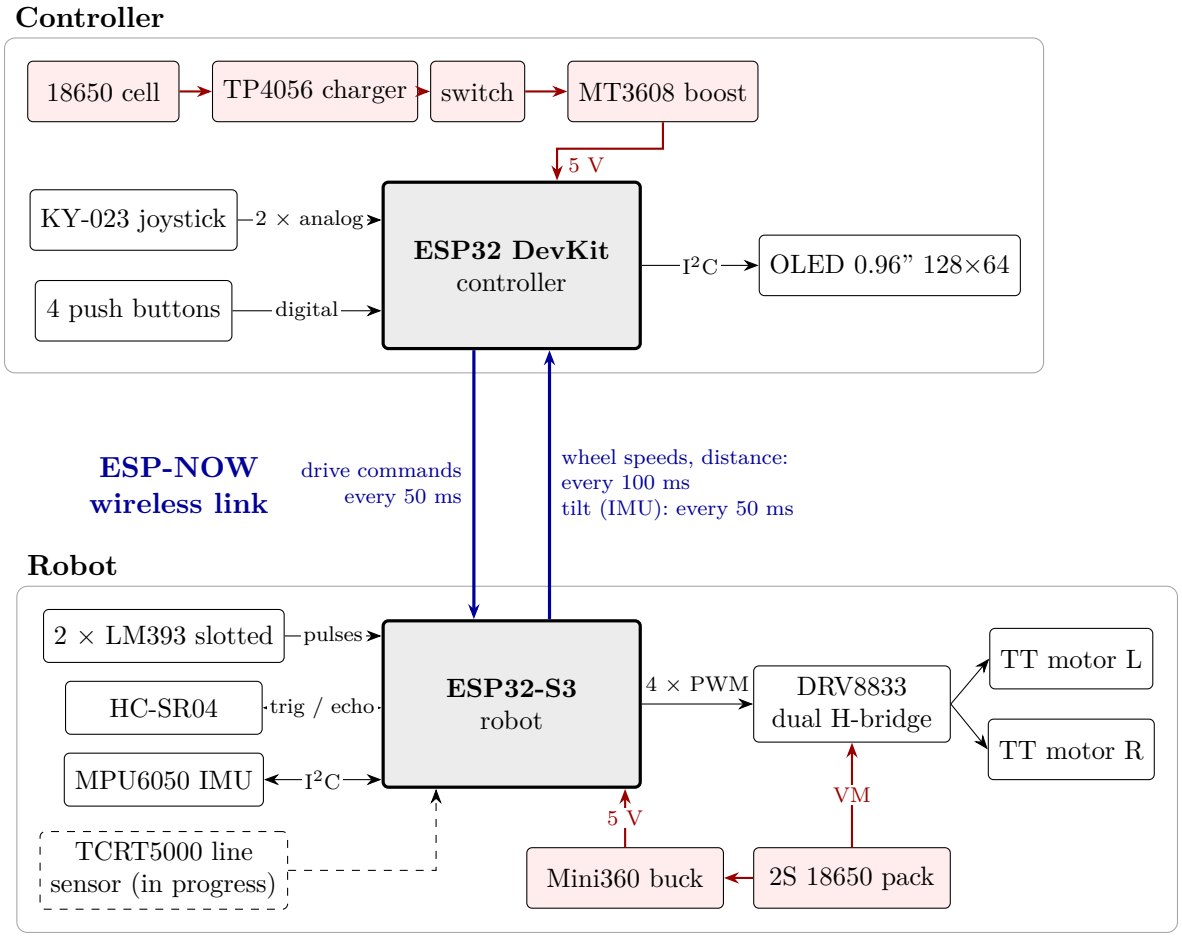


Figure 2.1: System architecture. The controller (top) and the robot (bottom) share no wire; over ESP-NOW the controller sends drive commands and the robot sends back its wheel speeds, the measured distance and its tilt. Red arrows are power, black arrows are signals, and the dashed block is in progress.

Table 2.2: Main features of the ESP32-S3 and of the N16R8 module used on the robot, whose figures follow Espressif's WROOM-1 N16R8 [1], [2].

Feature	Value
Processor	Dual-core Xtensa 32-bit LX7, up to 240 MHz
Memory	512 KB SRAM, 384 KB ROM (chip); 16 MB flash, 8 MB PSRAM (module)
Radio	2.4 GHz Wi-Fi (IEEE 802.11b/g/n), Bluetooth 5 (LE)
GPIOs	36 on the module, of which GPIO35–37 are taken by the PSRAM
Analog inputs	Two 12-bit SAR ADCs, up to 20 channels; ADC2 cannot be used while Wi-Fi is active
PWM	LED PWM controller, up to 8 channels
Supply	3.0–3.6 V (module); 5 V on the development board, regulated to 3.3 V

Pins that cannot be used freely. Not all 36 GPIOs are available. With an octal PSRAM, GPIO35, GPIO36 and GPIO37 carry the memory's data lines and cannot be used for anything else [1], [2], [3], even though our board still brings them out to its header. GPIO0, GPIO3, GPIO45 and GPIO46 are strapping pins, read by the chip at reset to choose its boot mode, so nothing connected to them should pull them high or low at power-up. GPIO19 and GPIO20 carry the USB interface and GPIO43 and GPIO44 the serial port used for programming [1]. Finally, because the robot uses its radio all the time, analog sensors must be wired to ADC1 pins (GPIO1 to GPIO10): the ADC2 channels cannot be read while Wi-Fi is running [1]. This matters for the five-channel line sensor, which gives analog outputs.

Table 2.3 gives the pins the robot uses at the time of writing. The DRV8833's sleep input is tied to 3.3 V, so the driver is always enabled and needs no pin.

Table 2.3: GPIO assignment on the robot (firmware file `pins_esp32s3.h`).

GPIO	Connected to	Note
13, 16	DRV8833 AIN1, AIN2	left motor, PWM
18, 17	DRV8833 BIN1, BIN2	right motor, PWM; the pair is swapped in firmware so that both wheels turn forward together
40	left LM393 sensor	digital pulses, read by interrupt
41	right LM393 sensor	digital pulses, read by interrupt
11	HC-SR04 TRIG	10 µs trigger pulse
12	HC-SR04 ECHO	through a 1 kΩ/2 kΩ divider (Section 2.4.5), timed by interrupt
8, 9	I^2C SDA, SCL	MPU6050 at 0x68 (Section 2.4.6); reserved for more I^2C sensors

2.4.2 Motor driver: DRV8833

A microcontroller pin can supply only a few milliamperes, far less than a motor draws, and it cannot reverse the polarity of the voltage across the motor. Both jobs are done by a motor driver built around an *H-bridge*.

The H-bridge. An H-bridge is made of four switches arranged around the motor, as in Figure 2.2. Closing Q_1 and Q_4 connects the left terminal of the motor to the supply and the right terminal to ground, and the motor turns one way. Closing Q_3 and Q_2 reverses the voltage, and the motor turns the other way. Closing both low-side switches Q_2 and Q_4 shorts the motor terminals together, which brakes it, and opening all four leaves it free to coast. The two switches of the same leg (Q_1 and Q_2, or Q_3 and Q_4) must never be closed together, since that would short the supply to ground; the driver chip prevents this internally.

The DRV8833. The robot uses one DRV8833 from Texas Instruments, a chip that contains two complete H-bridges, so a single module drives both wheels. Each bridge is controlled by two logic inputs, `IN1` and `IN2`, following Table 2.4. Its main ratings are listed in Table 2.5.

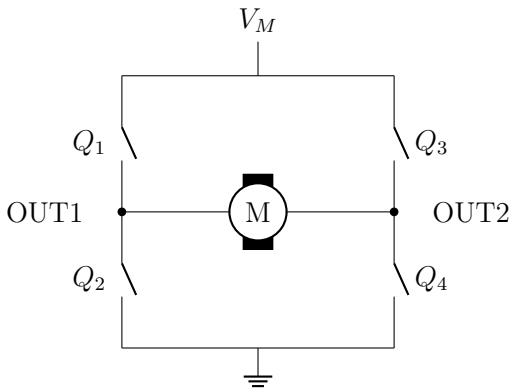


Figure 2.2: Principle of an H-bridge. The DRV8833 contains two of these, built from N-channel MOSFETs, one per motor [4].

Table 2.4: DRV8833 H-bridge logic, per bridge [4], and the switches of Figure 2.2 that it closes.

IN1	IN2	OUT1	OUT2	Switches closed	Function
0	0	Z	Z	none	Coast
1	0	H	L	Q_1, Q_4	Forward
0	1	L	H	Q_3, Q_2	Reverse
1	1	L	L	Q_2, Q_4	Brake

Table 2.5: Main ratings of the DRV8833 [4].

Parameter	Value
Motor supply voltage V_M	2.7 to 10.8 V
Output current per bridge (per motor)	1.5 A RMS, 2 A peak (HTSSOP package with exposed pad); 0.5 A RMS, 2 A peak (plain TSSOP package)
Undervoltage lockout (voltage)	below 2.6 V (V_M falling, maximum)
Overcurrent protection (current)	trips between 2 and 3.3 A; the bridge switches off and retries after 1.35 ms
Logic input voltage	high from 2 V (`nSLEEP` from 2.5 V), up to 5.75 V

Speed control by PWM. To vary the speed, the firmware does not hold an input high but switches it on and off rapidly, a technique called pulse-width modulation (PWM). If the input is high for a fraction D of each period, the duty cycle, the motor sees on average roughly $D \times V_M$, and because its winding and its inertia smooth the pulses, it turns as if fed that average voltage. The ESP32-S3 generates the PWM signals with its LED PWM controller, at 20 kHz with 8-bit resolution, so the duty cycle is set by a number from 0 to 255 (firmware files `pins_esp32s3.h` and `motor.cpp`). For forward motion the firmware applies PWM to `IN1` and holds `IN2` low; for reverse it does the opposite, and it sets both inputs low to stop. In the DRV8833's terms this is PWM with fast decay [4]: during the off part of each period the bridge is disabled and the motor coasts. Four PWM channels are therefore needed for the two motors.

Why the DRV8833. One chip drives both motors, which saves space and wiring on the waffle plate. Its supply range, 2.7 to 10.8 V, covers the whole range of a 2S lithium-ion pack, and its logic inputs accept the 3.3 V signals of the ESP32-S3 directly. It was also available locally at low cost.

The module we use is a small red breakout board that carries the chip and a 10 µF capacitor on the motor supply, and follows the pin layout of the common TB6612FNG boards, with the three pins the DRV8833 does not need marked NC. The chip's marking is not readable on our module, so we cannot tell from it whether it is the HTSSOP version, whose exposed pad allows 1.5 A per bridge, or the plain TSSOP one, limited to 0.5 A [4]. We therefore take the lower figure, 0.5 A continuous per motor, as the safe limit.

The driver we started with. The robot was first wired with two DRV8871 modules, one per motor. With them, the motors never turned at all, while the same motors ran with the DRV8833. We did not measure the battery voltage at the time, so we cannot state the cause with certainty. The DRV8871 datasheet points to a likely one: the chip is rated for a motor supply of 6.5 to 45 V, and its undervoltage lockout switches the bridge off when the supply falls to 6.1 V typically (6.4 V at most), only restarting once it rises back to about 6.3–6.5 V [5]. Our 2S pack is 7.4 V nominal (2 × 3.7 V, from the cell labels), only about 1 V above that threshold, and its voltage falls as it discharges and drops further for a moment each time a motor starts and draws a surge of current. A partly discharged pack could therefore keep the DRV8871 at or below its lockout threshold. The DRV8833, whose lockout is at 2.6 V, has no such problem with 2S.

2.4.3 Motors: TT gearmotors

Each wheel is driven by a TT gearmotor, the yellow plastic motor found in most hobby robot kits. It combines a small brushed DC motor with a plastic gearbox that reduces the speed and multiplies the torque, and has an output shaft on both sides of the gearbox. One side carries the wheel; the other is free, which is where the encoder disc of Section 2.4.4 is fitted.

Our motors carry no marking, and the shops we bought them from publish no specifications. We therefore use the figures published by Adafruit for the same common 1:48 TT motor as a reference, given in Table 2.6; they describe the type of motor, not measurements of our own units.

Three consequences follow for the design.

Table 2.6: Reference figures for the 1:48 TT gearmotor (vendor bench measurements [6], not measured on our motors).

	at 3 V	at 6 V
No-load speed (output shaft)	120 rpm	250 rpm
No-load current	150 mA	160 mA
Stall current	1.1 A	1.5 A
Stall torque	0.4 kg cm	0.8 kg cm

Voltage. The motor is rated for 3 to 6 V [6], while the 2S pack gives 7.4 V nominally and about 8.4 V when fully charged (4.2 V per cell, the usual full-charge voltage of a lithium-ion cell). The firmware therefore never applies full duty: drive commands are scaled so that the duty cycle does not exceed 204/255, or 80 %. On a pack at 7.7 V, the value used in the firmware comment, this gives about $0.8 \times 7.7 \approx 6.2$ V on average. When the pack is fully charged the average rises to about 6.7 V, slightly above the rating; this is a known limit of the current setting.

Current. While running, a TT motor draws well under 0.5 A, the safe limit we set for the driver in Section 2.4.2, but at stall, or for an instant when it starts, it can draw up to about 1.5 A at 6 V (Table 2.6). This is still below the 2 A peak the DRV8833 accepts and below its overcurrent trip point [4]. To soften the start-up surge, the firmware does not jump to the requested speed but ramps the duty cycle by 5 steps (out of 255) every 10 ms (`motor.cpp`), so going from rest to the 80 % cap takes about 0.4 s.

Speed. The wheels are standard 65 mm TT wheels (the diameter used in the CAD model and in the firmware; test T1 in Chapter 3 checks it in practice). One turn of the wheel moves the robot by $\pi \times 65 \approx 204$ mm. At the 250 rpm no-load speed of Table 2.6, this is about 0.85 m/s, or 3.1 km/h. Under the weight of the robot the real speed is lower, so the robot should not go much faster than about 3 km/h. This order of magnitude is a useful check on the speed the encoders report.

Why TT motors. They are geared, so they turn slowly enough to drive a wheel directly, cheap, easy to mount, and the dual shaft leaves room for an encoder. Above all they were the motors available to us.

2.4.4 Wheel speed sensors: LM393 slotted encoders

To report the speed of each wheel, the robot uses one slotted speed sensor per motor: a small board with a U-shaped optical fork, an LM393 comparator and a digital output.

Working principle. One arm of the fork holds an infrared LED and the other a phototransistor facing it. A plastic disc with evenly spaced slots is mounted on the free shaft of the TT motor so that its rim passes through the fork. As the wheel turns, the disc alternately lets the infrared light through a slot and blocks it, and the phototransistor's current rises and falls with it. This signal is slow and noisy at its edges, so the board feeds it to an LM393 comparator, which compares it with a fixed reference and gives a clean two-level output: one pulse per slot. The LM393's output stage is an open-collector

transistor that can only pull the line low, so the board carries a pull-up resistor that defines the high level [7]. We power the sensors from the ESP32-S3's 3.3 V supply, so that this high level is 3.3 V and the output can go straight to a GPIO.

From pulses to speed. With N slots on the disc and a wheel of diameter D, each pulse corresponds to a distance travelled by the wheel of

$$\Delta s = \frac{\pi D}{N}. \tag{2.1}$$

Our discs have $N = 20$ slots, counted on the real parts, and the wheels are 65 mm in diameter, which gives $\Delta s = \pi \times 65/20 \approx 10.2$ mm per pulse (`encoders.cpp`). The firmware first assumed 16 slots; this made every reported speed 25 % too high (20/16) until the slots were counted and the constant corrected. Each sensor output is wired to its own GPIO (40 and 41) and triggers an interrupt on every rising edge.

First method: counting pulses. The first version of the firmware counted the pulses of each wheel and, every 100 ms, computed the speed as

$$v = \frac{n\,\Delta s}{\Delta t}, \tag{2.2}$$

where n is the number of pulses counted during the interval $\Delta t = 100$ ms. This turned out to be too coarse. At the robot's speeds a 100 ms window holds only a few pulses, so one pulse more or less changes the result by 10.2 mm/0.1 s $\approx$ 102 mm/s, about 0.37 km/h, and the displayed speed jumped between neighbouring values.

Current method: timing the pulses. The firmware now measures the time between pulses instead. Each interrupt also records the period T_i since the previous pulse, taken from the microsecond timer. Every 100 ms, the speed is computed from the average period of the n pulses received in that window:

$$v_{\text{meas}} = \frac{n\,\Delta s}{\sum_{i=1}^{n} T_i}. \tag{2.3}$$

Since the periods are timed to the microsecond, the result no longer moves in steps of about 100 mm/s. Three details handle the edge cases. If no pulse arrives during a window, the time elapsed since the last pulse gives an upper bound on the speed, so the reading falls smoothly instead of dropping at once. If no pulse arrives for 300 ms, the wheel is considered stopped and the speed is set to zero; this means the slowest speed the robot can report is about 10.2 mm/0.3 s $\approx$ 34 mm/s. And the first pulse after a stop is not used, because its "period" is just the time the wheel was standing still. Finally, the measured value is smoothed with a first-order filter,

$$v_k = v_{k-1} + \alpha\,(v_{\text{meas}} - v_{k-1}), \qquad \alpha = 0.4, \tag{2.4}$$

where a larger α follows changes faster but lets more noise through.

The sensor has a single channel, so it sees that the wheel turns but not in which direction. The firmware takes the sign from the command it is applying to that motor, then sends both wheel speeds, in millimetres per second, to the controller every 100 ms.

Protection against false pulses. Slotted sensors of this kind can produce several rapid transitions as a slot edge passes, which would be counted as extra pulses. The interrupt routine therefore ignores any pulse that arrives less than 1 ms after the previous one on the same wheel. At the TT motor's no-load speed of 250 rpm (Table 2.6), a 20-slot disc gives about 83 pulses per second, one every 12 ms, so this 1 ms window cannot hide a real pulse.

Why these sensors. The board's fork fits around the rim of a disc on the TT motor's second shaft, the sensors cost little, and they were what we had. They give speed, not position or direction; a quadrature encoder would give both, at a higher cost.

2.4.5 Ultrasonic sensor: HC-SR04

At the front of the robot, an HC-SR04 ultrasonic module measures the distance to the nearest obstacle. It is mounted in a VBloc on the upper plate, facing forward.

Working principle. The HC-SR04 measures distance by time of flight. When its `TRIG` input receives a pulse of at least 10 µs, its transmitter sends a burst of eight ultrasonic cycles at 40 kHz. The sound travels to the obstacle, is reflected, and comes back to the receiver. The module raises its `ECHO` output when the burst leaves and lowers it when the echo returns, so the length t of the `ECHO` pulse is the round-trip time [8]. Since the sound covers the distance twice, the distance d to the obstacle is

$$d = \frac{v\,t}{2}, \tag{2.5}$$

where v is the speed of sound in air. The datasheet takes $v = 340$ m/s, which gives the rule of thumb $d\,[\text{cm}] = t\,[\mu\text{s}]/58$ [8]. The value of v depends on the air temperature, so the measured distance carries a small error when the air is much warmer or colder than assumed. Table 2.7 lists the module's main figures.

Table 2.7: Main figures of the HC-SR04 module, from the manufacturer's datasheet [8].

Parameter	Value
Supply	5 V, 15 mA
Range	2 cm to 4 m
Measuring angle	15°
Trigger	pulse of at least 10 µs
Ultrasonic burst	8 cycles at 40 kHz
Recommended cycle	at least 60 ms between measurements

Reading it without blocking. The maximum range sets how long an echo can take: for an obstacle at 4 m the sound travels 8 m, which takes about $8/340 \approx 24$ ms. A simple program would wait for the `ECHO` pulse to end, but waiting up to 25 ms in the main loop would delay the motor ramp and the failsafe, which run every 10 ms. The robot's firmware therefore times the echo with an interrupt on GPIO 12: the rising edge stores the start time, the falling edge the pulse width, and the loop never waits. Every

100 ms the firmware converts the last pulse width into a distance with Equation (2.5), using $v = 343$ m/s (0.343 mm/µs), sends it to the controller in millimetres, and fires the next ping. A pulse longer than 25 ms, about 4.3 m, is reported as "no object". The 100 ms spacing also respects the datasheet's advice to leave at least 60 ms between measurements, so that a late echo is not mistaken for the next one.

Level shifting. The HC-SR04 runs from 5 V, and its `ECHO` output swings to that supply. The ESP32-S3, however, accepts at most its own supply plus 0.3 V on an input, about 3.6 V [1], so `ECHO` cannot be connected directly. A voltage divider of two resistors brings it down: R_1 in series between `ECHO` and GPIO 12, and R_2 from GPIO 12 to ground. The pin then sees

$$V_{\text{GPIO}} = V_{\text{ECHO}} \frac{R_2}{R_1 + R_2}. \tag{2.6}$$

We use $R_1 = 1$ kΩ and $R_2 = 2$ kΩ, so a 5 V echo gives 3.33 V, within the input limit and well above the $0.75 \times V_{DD}$, about 2.5 V, that the ESP32-S3 needs to read a high level [1]. The `TRIG` input is left without conversion. The datasheet only asks for a "TTL pulse" [8] and gives no threshold, but in practice the module triggers reliably on the 3.3 V pulse from GPIO 11.

Why the HC-SR04. It is a cheap, widely sold module that is easy to drive with two pins, and it was available. Its limits are stated by the manufacturer: the beam spreads over about 15°, and reliable readings need a target of at least 0.5 m^2 with a fairly smooth surface [8].

2.4.6 Inertial sensor: MPU6050

The robot also carries an MPU6050 inertial measurement unit (IMU), which tells it how it is tilted and how fast it turns. At this stage the data is sent to the controller and displayed; it is the basis for later uses such as holding a straight line or detecting that the robot has tipped over.

What it measures. The MPU6050 from InvenSense combines a three-axis MEMS gyroscope, which measures angular rate, and a three-axis MEMS accelerometer, which measures acceleration including gravity, in one chip [9]. It runs from 2.375 to 3.46 V and is read over I^{2}C at up to 400 kHz, at the address 0x68 when its AD0 pin is low [9]. On the robot it shares an I^{2}C bus, on GPIO 8 (SDA) and GPIO 9 (SCL), that is reserved for later I^{2}C sensors as well **[TODO: module type (GY-521?) and which supply pin it is on]**.

Configuration. At start-up the firmware wakes the chip by clearing the `SLEEP` bit of its power-management register, and selects the most sensitive ranges: ±2 g for the accelerometer and ±250°/s for the gyroscope [10]. In these ranges one unit of the 16-bit raw value is 1/16 384 g and 1/131°/s [9], which are the scale factors the firmware divides by. It then averages 200 gyroscope readings while the robot stands still and stores them as offsets, because a real gyroscope does not read exactly zero at rest; the robot must therefore not be moved for about a second after it is switched on.

From raw data to tilt. Two estimates of the tilt are available, and each is good where the other is weak. The accelerometer gives the roll and pitch angles directly from the direction of gravity,

$$\phi_{\text{acc}} = \operatorname{atan2}(a_y, a_z), \qquad \theta_{\text{acc}} = \operatorname{atan2}\Big(-a_x, \sqrt{a_y^2 + a_z^2}\Big), \tag{2.7}$$

but this estimate is disturbed every time the robot accelerates or vibrates. The gyroscope, integrated over time, follows fast changes cleanly but slowly drifts away from the true angle. The firmware blends the two with a complementary filter, run on every pass of the main loop:

$$\phi_k = 0.96\,(\phi_{k-1} + \omega_x\,\Delta t) + 0.04\,\phi_{\text{acc}}, \tag{2.8}$$

and the same for the pitch θ with ω_y. The gyroscope term dominates over short times, while the small accelerometer share slowly pulls the estimate back and cancels the drift. Every 50 ms (20 Hz) the robot sends the roll, the pitch and the three angular rates to the controller, which shows them on its MPU6050 screen.

Why the MPU6050. It is a common and cheap module that gives six axes over two wires, and it was available to us. The yaw angle, the heading, cannot be corrected the same way, because gravity gives no reference for it; it would need a magnetometer, and the firmware does not compute it.

2.4.7 Power supply

The battery. The robot runs from two 18650 lithium-ion cells, the cylindrical 18 mm by 65 mm format used in laptop packs and power banks. Ours are METAMA cells labelled 3.7 V and 2500 mAh; we found no datasheet for them, so the label values are all we use. The two cells sit in a two-slot holder and are connected in series, the positive terminal of one to the negative terminal of the other, which makes a 2S pack:

- nominal voltage $2 \times 3.7 = 7.4$ V, rising to about 8.4 V when fully charged (4.2 V per cell, the usual full-charge voltage of a lithium-ion cell);
- capacity 2500 mAh, since cells in series share the same current;
- nominal energy about $7.4\ \text{V} \times 2.5\ \text{Ah} \approx 18.5$ Wh.

How long the robot runs on one charge is measured in Chapter 3 (test T2) rather than estimated here. The cells are removed from the robot and charged in a separate two-slot lithium-ion charger.

Distribution. The pack feeds two branches, as shown in Figure 2.1. The first goes straight to the motor supply pin V_M of the DRV8833, whose 2.7 to 10.8 V range covers the whole 2S range (Section 2.4.2). The second goes to a step-down converter that produces the 5 V for the ESP32-S3 board, whose own regulator then makes the 3.3 V the module and the LM393 sensors use. The HC-SR04, which needs 5 V, takes it from the board's 5 V pin, that is from the converter's output (`pins_esp32s3.h`).

The step-down converter. A pack that reaches 8.4 V cannot be connected to the board's 5 V pin directly. A linear regulator would turn the difference into heat; a switching *buck* converter instead switches the input on and off at high frequency and smooths the result with an inductor and a capacitor, so it wastes far less. We use a Mini360 module, a very small buck board built around the MP2307 controller, whose output voltage is set with an on-board trimmer. Before the ESP32-S3 was connected, we fed the module 7.4 V from a bench power supply and turned the trimmer until a multimeter read 5 V at its output. Its main figures, from a module description [11], are an input of 4.75 to 23 V, an output of 1 to 17 V, and 1.8 A continuous (3 A peak). The ESP32-S3 draws at most about 340 mA, while transmitting over Wi-Fi [1], so the converter works far below its limit.

What is missing. The pack has no battery management system (BMS) and no fuse, and the main switch is planned but not fitted yet. A BMS would cut the load when a cell is over-discharged and stop overcharging, and a fuse would protect the wiring from a short circuit; without them, these protections rely on the user, who must not run the pack flat and must charge the cells in the dedicated charger. One of our cells failed during development and was replaced; we did not identify the cause. Adding a 2S BMS, a fuse and the switch is the first item of future work on the hardware.

Decoupling is also minimal. TI asks for at least 10 µF close to the DRV8833's V_M pin, which the module provides, and recommends adding a local bulk capacitor, rated above the supply voltage, whose size is to be found by testing [4]. No bulk capacitor has been added yet, nor the small ceramic capacitors often soldered across a DC motor's terminals to absorb the noise of its brushes. Both are cheap to add: a 470 µF, 16 V electrolytic capacitor across the driver's V_M and GND pins, and a 0.1 µF capacitor on each motor.

2.5 The controller

The controller is the handheld unit the user drives the robot with. It reads a joystick and four buttons, shows menus and the robot's data on a small screen, and talks to the robot over ESP-NOW. Its parts are shown in the upper half of Figure 2.1, and its pin assignment in Table 2.8.

2.5.1 Microcontroller: ESP32 DevKit

The controller uses a classic ESP32, the predecessor of the chip on the robot. Its main figures are a dual-core Xtensa LX6 processor at up to 240 MHz, 520 KB of SRAM, 2.4 GHz Wi-Fi (IEEE 802.11b/g/n), Bluetooth 4.2, two 12-bit analog-to-digital converters and 34 GPIOs, six of which are inputs only [12]. The board is a generic 30-pin development board of the "ESP32 DevKit V1" type, with a USB-C connector and a CP2102 USB-to-serial converter for programming. Its module carries no marking, so we cannot name it exactly; like the robot's board, it takes 5 V on its `VIN` pin and regulates it to 3.3 V.

Why an ESP32. The controller's job is light: read a few inputs, draw a 128 × 64 screen and exchange small packets with the robot. A classic ESP32 does this easily and it is the board we already had. Since it uses the same radio and the same ESP-NOW protocol as the ESP32-S3, the two ends speak to each other without any extra hardware.

A smaller and cheaper chip, such as the single-core ESP32-C3, would probably be enough and is left as a possible improvement.

Table 2.8: GPIO assignment on the controller (firmware file `Bloc-Con/include/config.h`).

GPIO	Connected to	Note
34, 35	joystick X, Y	ADC1, input-only pins
12, 13, 14, 27	buttons UP, DOWN, FORWARD, BACK	to ground, internal pull-ups; 12 is a strapping pin (see text)
21, 22	OLED SDA, SCL	I^2C

A pin to handle with care. GPIO 12 is one of the ESP32's strapping pins: its level at reset, through the bit called MTDI, selects the voltage of the flash memory. Its default, set by an internal pull-down, is 0, which keeps the 3.3 V flash supply [12]. The UP button is connected to it, which is safe because the button can only pull the pin to ground and the firmware enables the internal pull-up only after boot. An external pull-up on this pin, however, could select the wrong flash voltage and stop the board from starting.

2.5.2 Human interface: joystick, buttons and display

Joystick. The KY-023 module is a two-axis thumb joystick: each axis moves the wiper of a potentiometer connected between the supply and ground, so the wiper voltage varies with the position of the stick and rests near mid-supply when the stick is released. The module also has a push switch, which the controller does not use. Although its supply pin is labelled +5 V, we power it from 3.3 V so that its outputs never exceed what the ESP32's inputs accept. The two wipers go to GPIO 34 and 35, which belong to ADC1. This matters because the radio is on all the time and ADC2 is shared with the Wi-Fi driver [13], while ADC1 is not. The ADC converts each voltage into a 12-bit number, from 0 to 4095. Its accuracy is limited: the datasheet gives a non-linearity of up to 12 LSB and advises averaging several samples [12], which is what the firmware does (Chapter 3).

Buttons. Four push buttons, UP, DOWN, FORWARD (select) and BACK, navigate the menus. Each connects its GPIO to ground when pressed. The pins use the ESP32's internal pull-up resistors, so no external resistor is needed: a released button reads high, a pressed one reads low.

Display. The screen is a 0.96-inch monochrome OLED of 128 × 64 pixels, driven by a Solomon Systech SSD1306 controller. It is connected over I^2C on GPIO 21 (SDA) and GPIO 22 (SCL), at the address 0x3C, the one the SSD1306 answers to when its address-select pin is low [14], and is powered from 3.3 V, within the controller's 1.65 to 3.3 V logic supply range [14]. The firmware draws each screen in a frame buffer in the ESP32's memory, $128 \times 64/8 = 1024$ bytes, and sends it to the display in one transfer. OLED pixels emit their own light, so the screen needs no backlight. During development we found the small 0.96-inch screens hard to read at arm's length, so a larger 2.42-inch display of the

same resolution, driven by the closely related SSD1309, is planned. The firmware already supports it through a single compile-time switch (`USE_SSD1309`), and the wiring is the same.

2.5.3 Power supply

The controller is powered by a single 18650 cell, a QHTC cell labelled 2000 mAh and 4.2 V. It is a cheaper cell than the robot's, which is acceptable because the controller draws far less current than two motors. The power chain has three stages, shown in Figure 2.1.

Charging and protection: TP4056 module. The cell stays in the controller and is charged through a TP4056 module with a USB-C input. The TP4056 is a linear charger for single lithium-ion cells. It charges at constant current, set by a programming resistor, then at constant voltage once the cell reaches 4.2 V (within 1.5 %), and charges a deeply discharged cell gently until it rises above 2.9 V [15]. Our module carries no reference; it is identical in layout to a USB-C TP4056 module sold locally, whose listing we use for its figures [16]: a maximum charge current of 1000 mA.

Unlike the robot's pack, the controller's cell is protected. The module has separate terminals for the battery (B+, B−) and for the load (OUT+, OUT−), and according to that listing it disconnects the load when the cell falls to 2.5 V or when the current exceeds 3 A [16]. The power switch is placed on the load side, after the charger, so the cell can be charged with the controller switched off.

Boost: MT3608. A lithium-ion cell stays below 4.2 V even when fully charged, under the 5 V the ESP32 board expects on its `VIN` pin. A step-up (boost) converter raises it. The MT3608 accepts 2 to 24 V at its input, switches at 1.2 MHz and regulates its output so that its feedback pin sits at 0.6 V, which gives

$$V_{\mathrm{OUT}} = 0.6\ \mathrm{V} \times \left(1 + \frac{R_1}{R_2}\right), \tag{2.9}$$

where R_1 and R_2 form the divider from the output to the feedback pin [17]. On the module, R_1 is a trimmer. The datasheet allows outputs up to 28 V, so the trimmer must be set to 5 V with a multimeter before the ESP32 is connected.

Load. The ESP32 draws up to about 240 mA while transmitting over Wi-Fi at full power [12], and the screen and joystick add little to this. A boost converter draws more current from the cell than it delivers, by the ratio of the voltages and its efficiency, so the controller's runtime on one charge is measured rather than estimated (Chapter 3).

2.6 Wireless link: ESP-NOW

The controller and the robot communicate over ESP-NOW. This section explains what the protocol is, why it was chosen, and what its use in BLOC-BOT does and does not guarantee. The exact packet formats are given with the firmware in Chapter 3.

What ESP-NOW is. ESP-NOW is, in Espressif's words, "a kind of connectionless Wi-Fi communication protocol that is defined by Espressif" [18]. It uses the same 2.4 GHz radio as Wi-Fi, but instead of joining a network, a device simply sends short frames addressed to another device's MAC address, or to the broadcast address FF:FF:FF:FF:FF:FF so that every listening device receives them. No access point and no connection set-up are needed. A frame carries at most 250 bytes of data in version 1.0 of the protocol (1470 bytes in version 2.0), and a device can register up to 20 peers [18]. On reception, the firmware is notified by a callback function that receives the sender's address and the data.

Why ESP-NOW and not Wi-Fi or Bluetooth. The link has a simple job: carry a few bytes of commands one way and a few bytes of measurements the other, several times per second, between two boards that are both Espressif chips.

- **Wi-Fi** in the usual sense would require one board to act as an access point and the other to join it before any data could flow, and then an IP-based protocol on top. This is more set-up than two boards exchanging small packets need.
- **Bluetooth**: the two chips do not even share the same versions. The controller's ESP32 supports Bluetooth 4.2 in both its Classic and Low Energy forms [12], but the robot's ESP32-S3 supports only Bluetooth 5 Low Energy [1]. A Bluetooth LE link would work, but it needs a connection between a central and a peripheral device and a description of the exchanged data as services and characteristics.
- **ESP-NOW** needs neither an access point nor a connection: each board can send as soon as the radio is started, and the same few lines of code run on both chips. The 250-byte limit is far above what BLOC-BOT needs, since its largest packet is 11 bytes.

The choice was also practical: the protocol was suggested to us during development, and it suits a system in which every board is an ESP32.

How BLOC-BOT uses it. Both boards send to the broadcast address, without pairing and without encryption, on the radio's current channel. Every packet starts with a one-byte identifier, and the receiver accepts a packet only if both this identifier and the packet's length match what it expects; anything else is ignored. The controller sends drive commands every 50 ms while its Drive screen is open, and the robot sends its wheel speeds and the measured distance every 100 ms and its tilt every 50 ms.

What the link does not guarantee. Three limits follow from these choices. The first is handled by the design; the other two remain open.

- **Delivery.** A successful send only means that the frame was received at the radio's MAC layer; Espressif states that reception by the application is not guaranteed [18]. BLOC-BOT therefore does not rely on any single packet: commands are repeated every 50 ms, and the robot stops its motors if none arrives for 300 ms, that is after about six missed commands in a row (requirement FR3).
- **Security and interference.** Broadcast frames cannot be encrypted [18], and any ESP-NOW device in range that sends a packet with the same identifier and length

would be obeyed. Two BLOC-BOTs used side by side would therefore interfere. Pairing each controller with its robot by MAC address, and enabling ESP-NOW's encryption, which is available for paired devices [18], would remove this limit and is left as future work.

- **Range.** We have not measured the range of the link, and the documentation we used gives no figure for it, so the thesis states none.

2.7 The modular platform

Plan: Written last, short. Waffle plate (210 x 210 x 10 mm PETG, hole pattern 3.2/4.5/6.0 mm), blocs HBloc / VBloc / HBloc_L, pillars. Old plate (printed) vs new plate (designed): print time and mass. One or two figures, the print cost table.

2.8 Conclusion

Plan: Short bridge to Chapter 3 (realisation and tests).

Chapter 3

Realisation and tests

3.1 Introduction

Chapter 2 chose the components and explained how each one works. This chapter describes how they were put together: the wiring of the two boards, the firmware that runs on them, the problems met along the way and how they were solved, and the tests that show what the finished system does. It ends with the cost of the platform.

3.2 Wiring

Both boards are built from ready-made modules connected with jumper wires, so their schematics are drawn at module level, in KiCad: each module appears as a box carrying the pins of its header, and connections are made by giving the same net name to the pins that are joined, a common convention that keeps a schematic readable when many wires would cross. Only the microcontroller pins in use are shown. The robot's schematic is given in Figure 3.1 and the controller's in Figure 3.2; their pin assignments are those of Tables 2.3 and 2.8. KiCad's electrical rules check reports no unconnected pin or dangling net on either sheet.

Three points of the robot's wiring deserve a comment. The DRV8833's `STBY` pin is tied to 3.3 V, so the driver is always enabled and no GPIO is spent on it, and its three NC pins, kept from the TB6612FNG board layout, are left open. The right motor's inputs are wired to GPIO 17 and 18 in that order but swapped in the firmware, so that both wheels turn forward for the same command; changing two constants was simpler than rewiring. Finally, the HC-SR04's `ECHO` line reaches GPIO 12 through the 1 kΩ/2 kΩ divider of Section 2.4.5.

On the controller, the cell is connected to the battery terminals (B+, B−) of the TP4056 module and everything else to its protected output (OUT+, OUT−), so the protection of Section 2.5.3 covers the whole controller. The power switch sits between that output and the MT3608, and the four buttons go straight from their GPIO to ground.

Plan: Photos of the wired robot and controller. **[TODO: photos]**

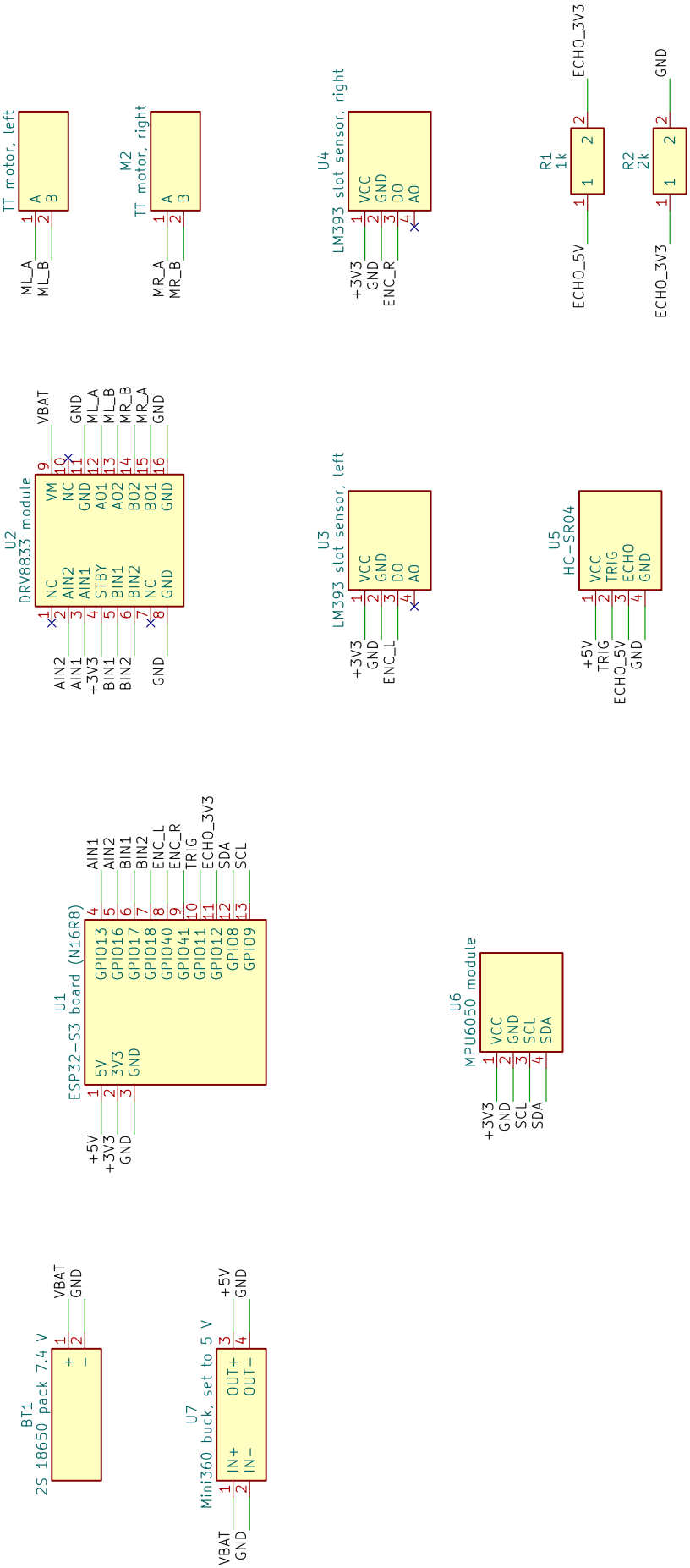


Figure 3.1: Schematic of the robot, drawn in KiCad. Modules are drawn as boxes with their header pins; pins carrying the same net name are connected. Only the ESP32-S3 pins in use are shown. Not shown: the main switch, a BMS and a fuse, which are not fitted yet. **[TODO: MPU6050 supply pin to confirm (drawn on 3V3)]**

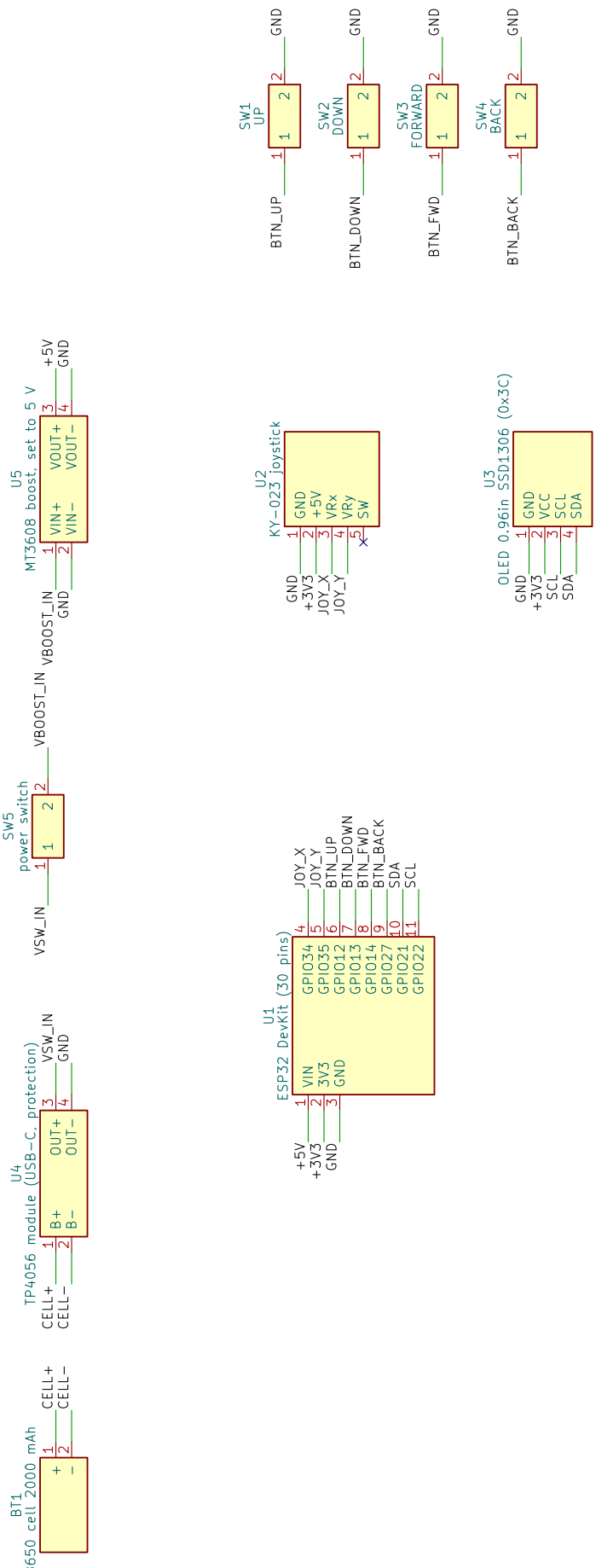


Figure 3.2: Schematic of the controller, drawn in KiCad, with the same conventions as Figure 3.1. The joystick is powered from 3.3 V although its pin is labelled +5V. The HC-SR04 wired to the controller during bench tests is not shown, since the sensor now sits on the robot.

3.3 Firmware

3.3.1 Tools

Both boards are programmed in C++ with the Arduino framework for Espressif chips, inside PlatformIO, an open build system that fetches the toolchain and the libraries a project declares and builds and uploads the firmware. Each board has its own project: the robot's at the root of the repository, the controller's in `Bloc-Con`. The controller also uses the U8g2 graphics library to drive its display. All the source code is kept in a single Git repository, whose history is also the record of what was changed and when. **[TODO: versions: Arduino-ESP32 core, PlatformIO platform; was the controller ever run in the Wokwi simulator (a Wokwi configuration exists in Bloc-Con)?]**

The code of each board is split into one file per function, listed in Table 3.1, so that each part can be read, tested and changed on its own. The packet formats are defined in a single header, `packets.h`, kept identical in both projects so that the two ends always agree on them.

Table 3.1: Organisation of the firmware (`.cpp` file and its `.h` header).

File	Role
Robot	
`main`	start-up and the 10 ms main loop
`radio`	ESP-NOW set-up, sending, reception callback
`drive`	drive commands: reception, mixing, link indicator
`motor`	PWM output, soft start, failsafe
`encoders`	wheel-speed measurement
`ultrasonic`	HC-SR04, echo timed by interrupt
`imu`	MPU6050 and the complementary filter
`bench`	serial commands for bench tests
`pins_esp32s3.h`	pin assignment
Controller	
`main`	start-up and the main loop
`radio`	ESP-NOW set-up, drive commands, data received from the robot
`joystick`	calibration and reading
`ui`	buttons, menu, splash, About and Settings screens
`screen_drive`, `screen_ultrasonic`, `screen_mpu`	one file per screen
`display`, `config.h`	display object, pin assignment
`packets.h`	packet formats, shared by both projects

3.3.2 Communication protocol

The two boards exchange four kinds of packet over ESP-NOW (Section 2.6), listed in Table 3.2. Each is a C structure sent as raw bytes. Its first byte is an identifier, and the receiver accepts a packet only when both this identifier and the packet's length match one of the structures it knows. Values that are naturally fractional are scaled to integers to keep the packets small: distances are sent in millimetres and angles in tenths of a degree.

Table 3.2: ESP-NOW packets exchanged between the controller (Con) and the robot (Bloc). All multi-byte fields are signed 16-bit integers.

ID	Direction	Content	Size	Period
0xB1	Con → Bloc	forward and turn commands, each from −100 to +100 (8-bit)	3 B	50 ms
0xB2	Bloc → Con	left and right wheel speeds, mm/s, negative in reverse	5 B	100 ms
0xB4	Bloc → Con	roll and pitch (0.1°), angular rates about x, y, z (0.1°/s)	11 B	50 ms
0xB5	Bloc → Con	distance in mm, −1 when no object is in range	3 B	100 ms

The drive command is only sent while the controller's Drive screen is open. Leaving that screen with the BACK button sends three stop commands in a row, so that the robot stops at once instead of waiting for its failsafe. On the controller, a value received from the robot is shown only if it is less than 500 ms old; otherwise the screen shows "–" or a waiting message.

3.3.3 Robot firmware

Structure. The robot's firmware is a single loop that runs every 10 ms (Figure 3.3). Each pass handles the bench-test serial commands, applies the latest drive command, sends the wheel speeds, the inertial data and the distance when their period has elapsed, and finally updates the motors. Work that cannot wait for the loop is done in interrupt routines: the radio callback stores each drive command as it arrives, the two encoder interrupts time the wheel pulses (Section 2.4.4), and the echo interrupt times the ultrasonic pulse (Section 2.4.5). No function in the loop waits for a sensor, so the motor ramp and the failsafe always run on time.

From command to wheel speeds. The robot is driven like a tank: each wheel has its own motor, and turning comes from a difference in speed between them. The drive command carries a forward value f and a turn value t, both between −100 and +100. The firmware mixes them into a target for each wheel,

$$L = \mathrm{clamp}(f + t)\,\frac{204}{100}, \qquad R = \mathrm{clamp}(f - t)\,\frac{204}{100}, \tag{3.1}$$

where clamp limits the sum to ±100 and 204 is the PWM cap of Section 2.4.2. Pushing the stick straight ahead ($f = 100$, $t = 0$) drives both wheels forward at the cap; pushing it fully to the right ($f = 0$, $t = 100$) turns the wheels in opposite directions and the robot spins on the spot; any position in between gives a curve.

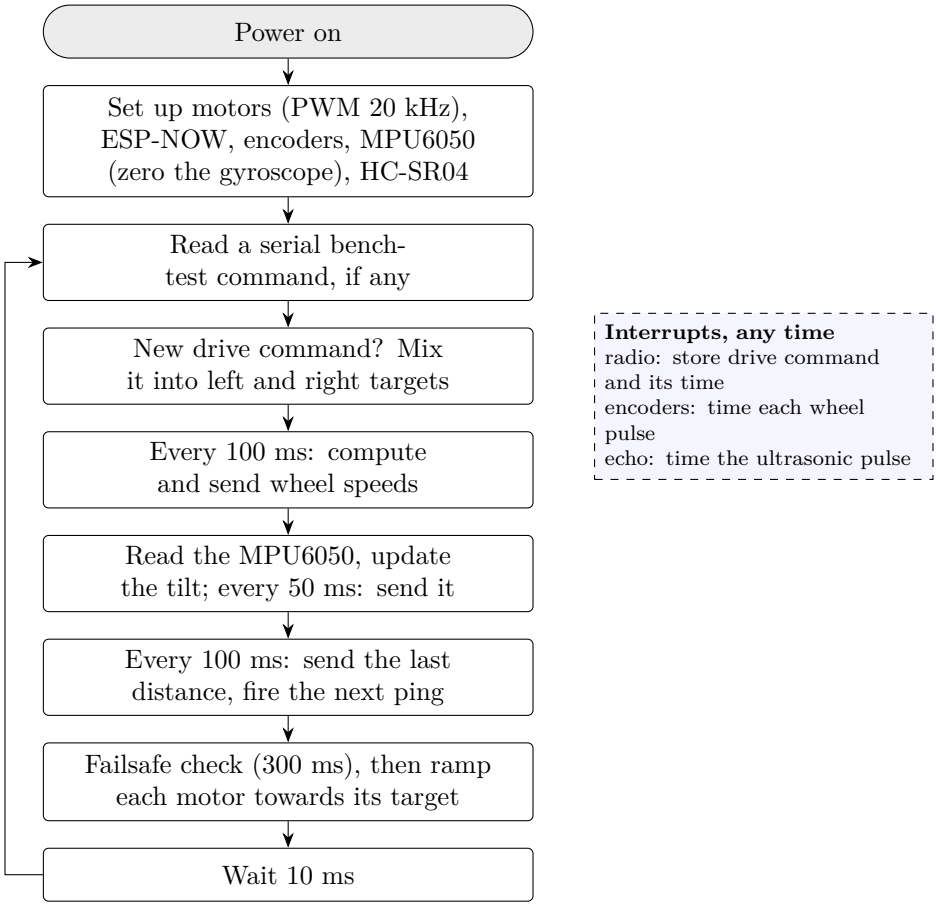


Figure 3.3: Main loop of the robot firmware. Dashed: work done in interrupt routines, outside the loop.

Soft start. The firmware does not apply a new target at once. Every 10 ms it moves each motor's duty cycle by at most 5 steps towards its target, so going from rest to the 80 % cap takes about 0.4 s, and so does a reversal from full forward to standstill. This limits the current surge when a motor starts (Section 2.4.7) and makes the robot's starts and stops smoother.

Failsafe. Every time a drive command is applied, the firmware records the time. If more than 300 ms pass without a new one, because the controller was switched off, left the Drive screen or went out of range, the firmware stops both motors immediately, without ramping, and they coast to a halt. This meets requirement FR3. With commands sent every 50 ms, the failsafe tolerates five lost packets in a row and acts on the sixth.

Bench-test interface. During development the robot can also be driven from a computer through its USB serial port: single letters drive forward, backward, turn, spin or stop, and `L` or `R` followed by a number sets one motor directly. A serial command runs for 2 s and then lets the failsafe stop the motors, so a robot on the bench never keeps running unattended. This interface is a tool for testing, not part of normal use.

3.3.4 Controller firmware

Screens. The controller's firmware is organised as a set of screens, only one of which is active at a time (Figure 3.4). It starts with an animated splash screen, then shows a menu with seven entries: Start, Settings, Ultrasonic, MPU6050, Drive, About and Reset. UP and DOWN move the selection, FORWARD opens the selected screen, and BACK returns to the menu from anywhere. Start does nothing yet and Settings only shows "Coming soon"; Reset restarts the controller. The main loop reads the buttons, redraws the active screen and waits 20 ms. A button press is ignored if it comes less than 180 ms after the previous one, which filters out contact bounce and stops a held button from scrolling too fast.

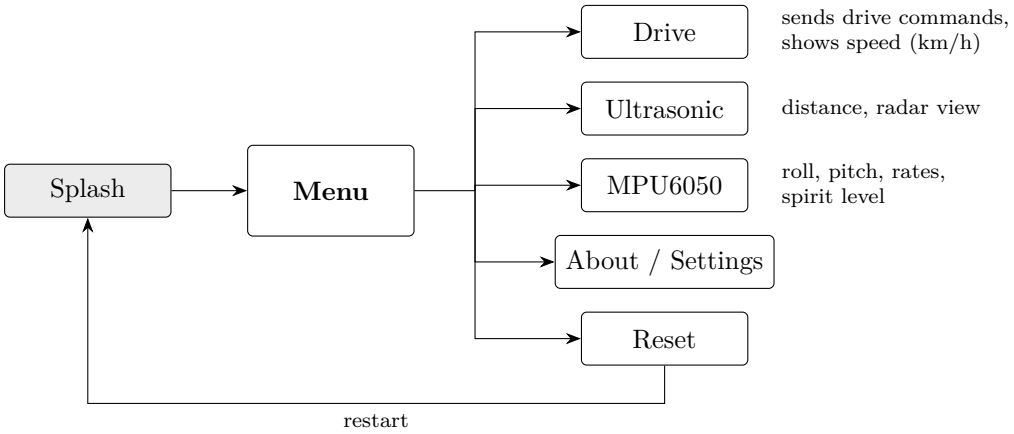


Figure 3.4: Screens of the controller. FORWARD opens the selected screen from the menu, and BACK returns to the menu from every screen.

Reading the joystick. At start-up, with the stick released, the firmware takes 50 readings of each axis over about a quarter of a second and stores their average as the

centre position. On the Drive screen, each axis is then read eight times and averaged, as the ESP32 datasheet advises for its converter [12]. The centre is subtracted, the result is divided by 20, and values smaller than 4 in magnitude, that is within 80 counts of the centre, are set to zero. This dead zone keeps the robot still when the stick rests slightly off centre. The forward and turn commands are finally scaled so that a value of 90 gives the full command of 100, and larger values are clipped. Since the end of the stick's travel corresponds to about $2048/20 \approx 102$, full speed is reached near the end of the travel, and the small margin makes sure it can always be reached. An earlier version used 30 instead of 90: full speed then came at less than a third of the travel, which made slow, precise driving difficult, so the scale was widened to use almost the whole stick.

Drive screen. The Drive screen draws a circle with a dot that follows the stick, the scaled X and Y values, and the robot's speed in kilometres per hour: the average of the two wheel speeds received from the robot, converted from mm/s by the factor 0.0036.

Ultrasonic and MPU6050 screens. The Ultrasonic screen prints the distance received from the robot and draws a radar-style view, in which the obstacle appears as a dot at the measured distance. The sweeping line of this view is only an animation: the sensor itself does not turn, so the angle at which the dot appears carries no information. The MPU6050 screen prints the roll, the pitch and the three angular rates, and draws a spirit level whose dot moves with the tilt. Both screens can also read a sensor wired directly to the controller, a fallback kept from bench testing that is used only when no data arrives from the robot; each screen shows in a corner which source it is using.

3.4 Problems met and solutions

Plan: Problem → cause → fix, one paragraph each, from the git history and the team's account: (1) DRV8871 motors never turned → likely UVLO on 2S (Section 2.4.2) → one DRV8833; (2) left motor would not run forward on GPIO 15 → a faulty jumper wire, found later → AIN1 moved to GPIO 13; (3) right motor turned backwards → wiring → the two pins swapped in firmware instead of rewiring; (4) speed reading jumped by about 100 mm/s → too few pulses per 100 ms window → period timing and filtering (Section 2.4.4); (4b) speeds 25 % too high → firmware assumed 16 slots, the discs have 20 → slots counted, constant corrected; (5) controller uploads stalled → the board's CP2102 at high baud rates → upload speed lowered to 115 200 baud (`platformio.ini`); (6) old waffle plate: 45 h print, 200 g → first print, no experience → lighter redesign (Section 2.7). **[TODO: confirm each story with the team; anything else that broke?]**

3.5 Tests and results

Plan: From `tracking/test_plan.md`. Each test: what was measured, how, the raw data in `results/<test>/`, a table or plot, and what it does and does not show. T1 real speed: 10 hand turns of a wheel must give 200 pulses (checks the 20 slots), then 2 m timed three times, measured vs displayed (explains or not the ∼10 km/h reading; expected order of magnitude about 3 km/h, Section 2.4.3). T2 runtime on a full charge. T3 ultrasonic: 10, 20, 50, 100 cm vs displayed. T4 print fits (screw hole 6.2 → 6.0, glue joint 0.14 → 0.20 mm per side). T5 print time, filament and robot mass. Demos D1 driving, D2 safety stop, D3 moving a bloc, D4 line following when it works. **[TODO: measurements]**

3.6 Cost

Printed parts. Table 3.3 gives the mass, print time and material cost of the printed parts. Masses and times are read from the slicer's G-code for each part, except the waffle plate in use, an early print for which only our own estimate is available. The cost is the mass of PETG at 4000 DA per kilogram, with and without a 5 % allowance for waste.

Table 3.3: Printed parts: mass, print time and PETG cost at 4000 DA/kg (`results/print_cost/print_cost.csv`).

Part	Mass (g)	Print time	Cost (DA)	+5 % waste
Motor brackets (left and right)	29.5	1 h 18	118	124
HBloc_L (line sensor holder)	28.0	1 h 15	112	117
VBloc	16.4	1 h 15	66	69
HBloc	13.9	0 h 39	56	59
Caster support	19.4	0 h 41	78	82
Waffle plate (first version, in use)[a]	200.0	45 h 00	800	840
Total	**307.2**	**50 h 08**	**1229**	**1290**

[a] Estimated by the team (about 200 g and 1 day 21 h); all other rows from the G-code.

Electronics.
Plan: Bill of materials with the prices actually paid (`results/bom_cost/paid_prices.csv`), marked paid vs shop listing, and the total cost of one robot and one controller. **[TODO: missing prices: MPU6050, TP4056 module, MT3608, OLED, cells for the controller, Mini360 and others bought by the team]**

3.7 Conclusion

Plan: What works against the requirements of Table 2.1, what the tests show, and what is left for the general conclusion.

Bibliography

[1] Espressif Systems, *ESP32-S3 Series Datasheet*, Version 2.2, Mar. 5, 2026. Accessed: Oct. 21, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

[2] Espressif Systems, *ESP32-S3-WROOM-1 / WROOM-1U Datasheet*, Version 1.8, Mar. 2, 2026. Accessed: Oct. 21, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

[3] Espressif Systems, *ESP32-S3-DevKitC-1 v1.1 User Guide*, Online documentation. Accessed: Oct. 21, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html

[4] Texas Instruments, *DRV8833 Dual H-Bridge Motor Driver*, SLVSAR1E, Datasheet, revised July 2015, 2015. Accessed: Oct. 10, 2026. [Online]. Available: https://www.ti.com/lit/gpn/DRV8833

[5] Texas Instruments, *DRV8871 3.6-A Brushed DC Motor Driver With Internal Current Sense*, SLVSCY9B, Datasheet, revised July 2016, 2016. Accessed: Oct. 21, 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/drv8871.pdf

[6] Adafruit Industries. "DC Gearbox Motor – "TT Motor" – 200RPM – 3 to 6VDC." Product 3777, technical details, Accessed: Oct. 21, 2026. [Online]. Available: https://www.adafruit.com/product/3777

[7] Texas Instruments, *LM393B, LM2903B, LM193, LM293, LM393 and LM2903 Dual Comparators*, SLCS005AH, Datasheet, revised April 2025, 2025. Accessed: Oct. 24, 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm393.pdf

[8] ElecFreaks, *Ultrasonic Ranging Module HC-SR04*, Datasheet, hosted by SparkFun Electronics. Accessed: Oct. 24, 2026. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[9] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification*, Revision 3.1, PS-MPU-6000A-00, Oct. 24, 2011. Accessed: Oct. 24, 2026. [Online]. Available: https://cdn.sparkfun.com/datasheets/Components/General%20IC/PS-MPU-6000A.pdf

[10] InvenSense Inc., *MPU-6000 and MPU-6050 Register Map and Descriptions*, Revision 4.0, RM-MPU-6000A-00, Mar. 9, 2012. Accessed: Oct. 24, 2026. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf

[11] Components101. “Mini360 DC-DC Buck Converter Module Pinout, Datasheet, Features, Specs and Applications,” Accessed: Oct. 24, 2026. [Online]. Available: https://components101.com/modules/mini360-dc-dc-buck-converter-module

[12] Espressif Systems, *ESP32 Series Datasheet*, Version 5.3. Accessed: Oct. 21, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[13] Espressif Systems. “Analog to Digital Converter (ADC) Oneshot Mode Driver – ESP32.” ESP-IDF Programming Guide v6.1, Accessed: Oct. 24, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc/adc_oneshot.html

[14] Solomon Systech, *SSD1306 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Rev 1.1, Apr. 2008. Accessed: Oct. 24, 2026. [Online]. Available: https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

[15] NanJing Top Power ASIC Corp., *TP4056 1A Standalone Linear Li-ion Battery Charger with Thermal Regulation in SOP-8*, Datasheet, hosted by SparkFun Electronics. Accessed: Oct. 24, 2026. [Online]. Available: https://dlnmh9ip6v2uc.cloudfront.net/datasheets/Prototyping/TP4056.pdf

[16] Esp32Lab. “Chargeur LiPo USB-C TP4056.” Product page (Algerian shop), in French, Accessed: Oct. 24, 2026. [Online]. Available: https://esp32lab.com/boutique/chargeur-lipo-usb-c-tp4056/

[17] Aerosemi Technology, *MT3608 High Efficiency 1.2MHz 2A Step Up Converter*, Datasheet, hosted by Olimex. Accessed: Oct. 24, 2026. [Online]. Available: https://www.olimex.com/Products/Breadboarding/BB-PWR-3608/resources/MT3608.pdf

[18] Espressif Systems. “ESP-NOW – ESP32.” ESP-IDF Programming Guide v6.1, Accessed: Oct. 25, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html